



資料結構

Data Structure

Lab 09

姓名：夏宇柔
學號：113840005

Lab09-Ex1.

Complete the following parts in the sample code: deleteNode
Add comments to the program to clearly describe its actions.
Illustrate the structure of the tree after each insertion and deletion.

Code

```
#include <iostream>
using namespace std;

// =====
// 樹的節點類別
// =====
class TreeNode {
public:
    int data;           // 節點儲存的資料
    TreeNode* left;     // 指向左子節點
    TreeNode* right;    // 指向右子節點

    // 節點建構子：設定資料並初始化左右指標為空
    TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};

// =====
// 二元搜尋樹類別
// =====
class BinarySearchTree {
private:
    TreeNode* root; // 根節點指標

    // --- 私有的遞迴插入方法 ---
    TreeNode* insert(TreeNode* node, int val) {
        if (node == nullptr) {
            return new TreeNode(val); // 找到插入位置，建立新節點
        }

        if (val < node->data) {
            node->left = insert(node->left, val); // 插入左子樹
        }
        else if (val > node->data) {
            node->right = insert(node->right, val); // 插入右子樹
        }
        // 如果 val == node->data, 不插入（避免重複）

        return node; // 回傳更新後的節點
    }

    // --- 私有的遞迴搜尋方法 ---
    bool search(TreeNode* node, int target) {
        if (node == nullptr) return false;

        if (node->data == target) return true;
        else if (target < node->data) return search(node->left, target); // 繼續在左子樹找
        else return search(node->right, target); // 繼續在右子樹找
    }
}
```

```

// --- 私有的中序遍歷方法 (左-根-右) ---
void inOrderTraversal(TreeNode* node) {
    if (node == nullptr) return;

    inOrderTraversal(node->left);    // 先走訪左子樹
    cout << node->data << " ";    // 印出目前節點的資料
    inOrderTraversal(node->right); // 再走訪右子樹
}

// --- 私有的刪除方法 ---
TreeNode* deleteNode(TreeNode* node, int val) {
    if (node == nullptr) return nullptr; // return nullptr if current
node is nullptr; empty tree
    if (val < node->data) { //如果要刪的值比目前這個節點小,往左子樹找
        node->left = deleteNode(node->left, val);
    }
    else if (val > node->data) { //如果要刪的值比目前這個節點大,往右子樹找
        node->right = deleteNode(node->right, val);
    }
    else { //如果要刪的值等於目前這個節點,進入刪除的三種情況
        if (node->right == nullptr && node->left == nullptr) { // no
children; delete directly
            delete node; // delete node to free memory
            return nullptr; // return nullptr
        }
        else if (node->left == nullptr) { // only has right child; replace
node with right subtree
            TreeNode* temp = node->right; // copy right sub-tree
            delete node; // delete node to free memory
            return temp; // return temporary copy to replace original
node
        }
        else if (node->right == nullptr) { // only has left child; replace
node with left subtree
            TreeNode* temp = node->left; // copy left sub-tree
            delete node; // delete node to free memory
            return temp; // return temporary copy to replace original
node
        }
        else { // has two children; find smallest of right sub-tree,
replace data & delete successor node
            node->data = findMin(node->right)->data; // find smallest of
right sub-tree & replace data
            node->right = deleteNode(node->right, node->data); //
recursively delete the duplicate node (successor) in right subtree
        }
    }
    return node;
}

// --- 私有的找最小值方法 ---
TreeNode* findMin(TreeNode* node) {
    while (node->left != nullptr) {
        node = node->left;
    }
    return node;
}

```

```

public:
    // --- 建構子：初始化為空樹 ---
    BinarySearchTree() : root(nullptr) {}

    // --- 公開插入方法 ---
    void insert(int val) {
        root = insert(root, val);
    }

    // --- 公開搜尋方法 ---
    bool search(int target) {
        return search(root, target);
    }

    // --- 公開中序遍歷方法 ---
    void displayInOrder() {
        cout << "樹的中序遍歷（排序後）結果：" << endl;
        inOrderTraversal(root);
        cout << endl;
    }

    // --- 公開刪除方法 ---
    void deleteValue(int val) {
        root = deleteNode(root, val);
    }
};

int main() {
    BinarySearchTree bst; // 建立一棵空的樹

    // 預設插入的資料
    int values[] = { 7, 1, 4, 3, 8, 13, 12, 10, 16, 5, 14, 17 };
    int n = sizeof(values) / sizeof(values[0]);

    // 把資料插入樹中
    for (int i = 0; i < n; ++i) {
        bst.insert(values[i]);
    }

    // 讓使用者輸入要插入的鍵值
    int key;
    cout << "請輸入要搜尋/新增的鍵值：" << endl;
    cin >> key;

    if (bst.search(key)) {
        cout << "值 " << key << " 已存在樹中，不新增。" << endl;
    }
    else {
        cout << "值 " << key << " 不存在，已新增進樹中。" << endl;
        bst.insert(key);
    }

    // 顯示目前樹的中序遍歷結果
    bst.displayInOrder();
}

```

```

// 讓使用者輸入要刪除的鍵值
int deleteKey;
cout << "請輸入要刪除的鍵值 : ";
cin >> deleteKey;

// 刪除節點
bst.deleteValue(deleteKey);

// 顯示刪除後的樹
cout << "刪除後的樹（中序遍歷） : " << endl;
bst.displayInOrder();

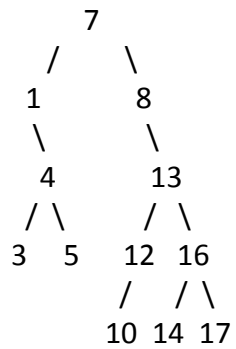
return 0;
}

```

Discussion Section

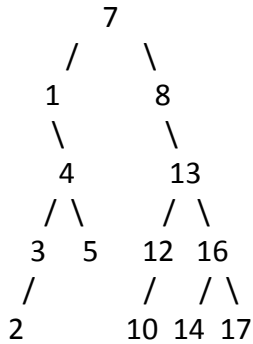
Original Tree:

```
int values[] = { 7, 1, 4, 3, 8, 13, 12, 10, 16, 5, 14, 17 };
```

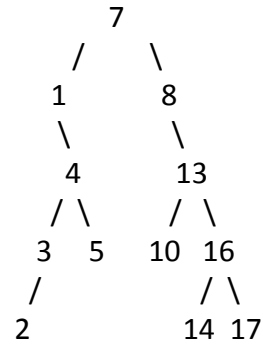


Test 1: Insert 2, Delete 12

Tree after Insert 2:



Tree after Delete 12:



Result Screenshot:

The screenshot displays the Visual Studio IDE with a C++ project named 'ConsoleApplication1'. The code implements a binary search tree with functions for insertion and deletion. The output window shows the program's execution, including prompts for inserting and deleting values, and the resulting in-order traversals.

```
60  TreeNode* deleteNode(TreeNode* node, int val) {
61      if (node == nullptr) return nullptr;
62      if (val < node->data) { //如果要刪除的
63          node->left = deleteNode(node->left, val);
64      }
65      else if (val > node->data) { //如果要刪除的
66          node->right = deleteNode(node->right, val);
67      }
68      else { //如果要刪除的值等於目前這個節點，就刪除
69          if (node->right == nullptr && node->left != nullptr) {
70              delete node; // delete node
71              return node->left; // return left child
72          }
73          else if (node->left == nullptr && node->right != nullptr) {
74              delete node; // delete node
75              return node->right; // return right child
76          }
77          else if (node->right == nullptr && node->left == nullptr) {
78              delete node; // delete node
79              return nullptr; // return nullptr
80          }
81          else { // has two children; find min in right subtree
82              TreeNode* temp = node->right;
83              while (temp->left != nullptr) temp = temp->left;
84              node->data = temp->data;
85              deleteNode(temp, temp->data);
86          }
87      }
88      return node;
89  }
```

Output:

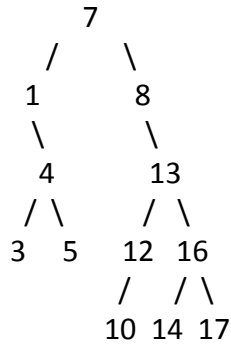
```
Build started at 10:34 PM...
----- Build started: Project: ConsoleApplication1, Configuration: Debug x64 -----
l>ConsoleApplication1.cpp
l>ConsoleApplication1.vcxproj -> C:\Users\User\source\repos\ConsoleApplication1\Debug\ConsoleApplication1.exe
***** Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped *****
***** Build completed at 10:35 PM and took 02.569 seconds *****
```

Microsoft Visual Studio Debug Console:

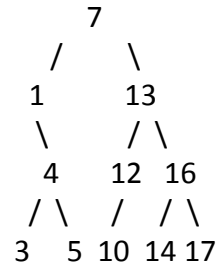
```
請輸入要搜尋/新增的鍵值: 2
值 2 不存在, 已新增進樹中。
樹的中序遍歷 (排序後) 結果:
1 2 3 4 5 7 8 10 12 13 14 16 17
請輸入要刪除的鍵值: 12
刪除後的樹 (中序遍歷):
樹的中序遍歷 (排序後) 結果:
1 2 3 4 5 7 8 10 13 14 16 17
C:\Users\User\source\repos\ConsoleApplication1\Debug\ConsoleApplication1.exe (process 15728) exited with code 0.
Press any key to close this window . . .
```

Test 2: Insert 8, Delete 8

Tree after Insert 8: No change, duplicate data



Tree after Delete 8:



Result Screenshot:

The screenshot shows the Visual Studio IDE with the `deleteNode` function in `ConsoleApplication1.cpp`. The function is designed to delete a node from a binary tree. It handles three cases: if the node to be deleted is the root, if it's a leaf node, and if it has children. The output window shows the results of the program execution.

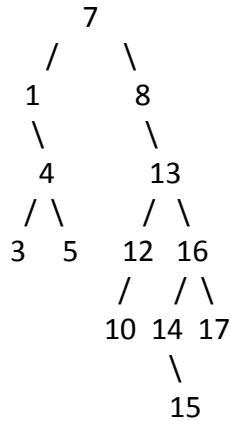
```
TreeNode* deleteNode(TreeNode* node, int val) {
    if (node == nullptr) return nullptr; //
    if (val < node->data) { //如果要刪的鍵值比目前節點小，就往左子樹找
        node->left = deleteNode(node->left, val);
    }
    else if (val > node->data) { //如果要刪的鍵值比目前節點大，就往右子樹找
        node->right = deleteNode(node->right, val);
    }
    else { //如果要刪的鍵值等於目前這個節點，進入刪除邏輯
        if (node->right == nullptr && node->left == nullptr) {
            delete node; // delete node to return nullptr; // return nullptr;
            return nullptr;
        }
        else if (node->left == nullptr) {
            TreeNode* temp = node->right;
            delete node; // delete node to return temp; // return temp;
            return temp;
        }
        else if (node->right == nullptr) {
            TreeNode* temp = node->left;
            delete node; // delete node to return temp; // return temp;
            return temp;
        }
        else { // has two children; find the min in the right subtree
            node->data = findMin(node->right);
            node->right = deleteNode(node->right, node->data);
        }
    }
}
```

Microsoft Visual Studio Debug Console Output:

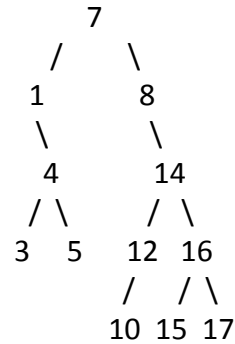
```
請輸入要搜尋/新增的鍵值：8
值 8 已存在樹中，不新增。
樹的中序遍歷（排序後）結果：
1 3 4 5 7 8 10 12 13 14 16 17
請輸入要刪除的鍵值：8
刪除後的樹（中序遍歷）結果：
1 3 4 5 7 10 12 13 14 16 17
C:\Users\User\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (process 20888)
Press any key to close this window . . .
```

Test 3: Insert 15, Delete 13

Tree after Insert 15:



Tree after Delete 13:



Result Screenshot:

